

DEVOPS MASTER COURSE

Volume 10C

GitHub Actions and GitLab CI/CD

Workflow Engineering, Reusable Automation, Runner Fleets, Identity, Security, Scaling, Governance, and Enterprise Platform Operations

Chapters 21-30 | 2026 Edition

From repository-level automation to a governed enterprise delivery platform

This volume treats hosted CI/CD as production infrastructure. Every workflow, runner, reusable component, credential path, deployment gate, and artifact must be versioned, isolated, observable, recoverable, and attributable to the exact source revision that produced it.

DevOps Master Course

Volume 10 Roadmap

Volume 10 contains six parts. Part C develops GitHub Actions and GitLab CI/CD from workflow syntax through reusable delivery products, secure runner fleets, protected environments, identity federation, scaling, governance, and cross-platform enterprise operations.

Part	Coverage	Status
10A	CI/CD Foundations and Pipeline Engineering	Complete
10B	Jenkins Administration and Enterprise Engineering	Complete
10C	GitHub Actions, GitLab CI/CD, Runners, Security, and Enterprise Operations	Current
10D	Kubernetes Delivery, GitOps, Argo CD, Flux, and Progressive Delivery	Planned
10E	DevSecOps, Supply-Chain Security, Policy, Observability, and Governance	Planned
10F	Enterprise Projects, Troubleshooting, Certifications, Interviews, and Final Capstone	Planned

Part C Learning Outcomes

- Engineer GitHub Actions workflows with explicit events, permissions, contexts, dependencies, concurrency, evidence, and failure behavior.
- Design GitHub-hosted, larger, self-hosted, and Kubernetes-based runner fleets with trust boundaries, autoscaling, cleanup, and capacity objectives.
- Create reusable workflows, composite actions, custom actions, and organization-level golden paths with versioned interfaces and dependency controls.
- Use OIDC, environments, deployment protection, artifact attestations, immutable references, and least-privilege tokens to secure GitHub delivery.
- Engineer GitLab pipelines with workflow rules, needs-based DAGs, includes, inputs, components, child pipelines, multi-project pipelines, and protected deployments.
- Operate GitLab Runner fleets with secure executors, autoscaling, cache controls, observability, upgrade governance, and incident containment.
- Build a cross-platform CI/CD platform product with policy, metrics, service objectives, migration controls, and a tested recovery model.

How to Use This Book

- Run labs in disposable repositories and non-production cloud accounts. Do not test fork-pull-request behavior, runner compromise, token scope, or deployment rollback against production systems.
- Use organization or group sandboxes when testing policies, runner groups, protected environments, reusable workflows, CI/CD components, and identity federation.
- Pin actions, components, images, and tools to reviewed versions or immutable digests. Record every resolved dependency in the lab evidence.
- Treat workflow code from external contributors as untrusted. Separate validation from privileged release paths and never expose high-value self-hosted runners to arbitrary pull requests.
- Publish immutable artifacts once, promote by digest, and preserve source revision, workflow revision, runner image, dependency lock, test evidence, SBOM, and attestation.

- Capture successful and failed paths. A CI/CD platform is not production-ready until runner loss, identity failure, canceled workflows, stale caches, artifact corruption, and failed deployments have been exercised.

Lab Safety and Evidence Standard

- Use synthetic secrets and dedicated cloud roles with minimal permissions. Revoke all temporary credentials after each identity lab.
- Record repository or project, commit SHA, workflow or pipeline revision, event type, actor, runner identity, environment, artifact digest, and deployment ID for every lab.
- Do not paste tokens, OIDC JWTs, masked variables, environment secrets, job tokens, runner registration tokens, or cloud credentials into reports.
- Separate public-fork validation runners from trusted release runners and document why each network path and credential exists.
- Retain sanitized logs, test reports, policy results, runner telemetry, deployment evidence, and rollback evidence outside ephemeral workspaces.
- Before destructive tests, define the stop condition, rollback command, responsible owner, and evidence required to prove recovery.

Chapter 21: GitHub Actions Architecture, Workflow Syntax, Events, Contexts, and Permissions

GitHub Actions combines repository events, YAML workflows, jobs, steps, actions, runners, contexts, tokens, artifacts, environments, and APIs. Reliable workflows begin with event and trust modeling, explicit permissions, deterministic dependencies, and evidence tied to the exact source revision.

Learning Objectives

- Explain workflow, event, job, step, action, runner, context, token, artifact, cache, and environment responsibilities.
- Design event filters and workflow-level controls that avoid duplicate, stale, or unauthorized runs.
- Use contexts, expressions, variables, outputs, and permissions without creating injection or privilege-escalation paths.
- Build a maintainable workflow with explicit failure, timeout, cancellation, and evidence behavior.

Core Concepts

Workflow and event: A workflow is a versioned YAML definition under `.github/workflows`. Events such as `push`, `pull_request`, `workflow_dispatch`, `schedule`, `workflow_call`, `release`, or `repository_dispatch` create runs. Event payloads are untrusted input and must not be interpolated directly into shell commands.

Jobs and steps: Jobs execute on runners and may depend on other jobs. Steps run actions or shell commands in one job workspace. Job boundaries provide isolation and parallelism but require explicit output, artifact, or external-state handoff.

Contexts and expressions: Contexts expose event, repository, actor, matrix, runner, job, step, environment, secret, and variable data. Expressions are evaluated before or during execution depending on the field. Treat user-controlled fields as data, not executable syntax.

GITHUB_TOKEN: GitHub creates a scoped token for each job. Set permissions explicitly at workflow or job level and grant only the capabilities required. Token scope and repository settings determine what the workflow can read or modify.

Action dependency: An action is executable supply-chain code. Reference trusted publishers, review source, pin to immutable commit SHAs for sensitive workflows, and use dependency review or policy to detect unauthorized changes.

Concurrency: Concurrency groups prevent conflicting runs and can cancel superseded work. Use `commit-`, `branch-`, `pull-request-`, `environment-`, or `service-aware` keys so validation can be replaced while stateful deployments remain serialized.

Evidence: A trustworthy run records source SHA, workflow file revision, event, actor, permissions, runner image, dependency versions, test reports, artifact digest, and deployment result. Logs alone are insufficient because retention and redaction can change.

Reference Workflow Skeleton

```
name: validate
on:
  pull_request:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
```

```

concurrency:
  group: validate-${{ github.event.pull_request.number || github.ref }}
  cancel-in-progress: true

jobs:
  test:
    timeout-minutes: 20
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@<reviewed-commit-sha>
      - name: Test
        run: ./ci/test.sh
      - name: Publish test evidence
        if: always()
        uses: actions/upload-artifact@<reviewed-commit-sha>
        with:
          name: test-evidence-${{ github.sha }}
          path: reports/
          retention-days: 14

```

Architecture and Operating Decisions

- Select the narrowest event and branch or path filters that satisfy the delivery requirement; avoid running privileged workflows on broad events.
- Declare permissions explicitly and prefer job-level elevation for the one job that needs it.
- Use repository scripts for complex logic so behavior can be tested outside the workflow engine.
- Cancel obsolete validation but serialize state-changing deployments using stable concurrency groups.
- Publish evidence even when a job fails, while avoiding secrets, tokens, or sensitive payloads in logs and artifacts.

Production Checklist

- Every event source and trust level is documented.
- Workflow and job token permissions are explicit.
- Third-party actions are reviewed and pinned.
- Timeout, cancellation, retry, and failure evidence behavior is defined.
- Checks are associated with the exact commit SHA being evaluated.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Workflow does not run	Event, branch, path, default-branch, or YAML parsing mismatch	Inspect the event payload and workflow location; validate filters against the actual ref and changed paths	Event JSON, workflow revision, filter decision
Workflow runs twice	Overlapping push and pull_request events or duplicate dispatch	Choose one authoritative validation trigger or deduplicate using event-aware concurrency	Run causes, event IDs, concurrency group
Permission denied from API	GITHUB_TOKEN permissions are missing or repository policy restricts them	Set the minimum required permission at job level and validate organization policy	Permissions block, API response, policy setting

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Shell command is manipulated	Untrusted context value was interpolated into run syntax	Pass values through quoted environment variables or files and validate allowed formats	Workflow diff, payload sample, sanitized command trace

Hands-on Lab 21: Create an Event-Safe GitHub Actions Validation Workflow

Prerequisites

- A disposable GitHub repository.
- A sample application with test and lint scripts.
- Permission to configure branch protection.
- A feature branch and pull request.

Implementation

1. Create a pull_request validation workflow with explicit contents: read permission, timeout, and concurrency cancellation.
2. Pin checkout and artifact actions to reviewed commit SHAs.
3. Move build and test logic into repository scripts and execute them locally before CI.
4. Publish test reports on success and failure and include the source SHA in the artifact name.
5. Add the workflow check to branch protection and prove that a newer commit cannot reuse a stale successful check.
6. Inject a suspicious pull-request title or branch name and prove it is handled as data rather than shell syntax.

Validation Evidence

- The workflow runs once per pull-request revision.
- Token permissions are minimal and visible in evidence.
- Artifacts contain reports but no secrets.
- The protected branch requires the check for the current commit.

Cleanup

- Delete the sample repository or remove branch protection.
- Retain only sanitized workflow and evidence examples.

Interview Questions and Model Answers

1. Why set GitHub Actions permissions explicitly?

The generated token can otherwise receive broader defaults than the job needs. Explicit permissions reduce blast radius, document intent, and make policy review possible.

2. When should validation runs be canceled?

Cancel runs superseded by a newer revision when they have no unique release or audit value. Do not cancel stateful deployment or migration work without a designed recovery model.

3. Why pin an action to a commit SHA?

Tags can move. A commit SHA makes the executed dependency immutable, although the organization must still monitor that pinned revision for vulnerabilities and update it deliberately.

4. What makes a workflow check trustworthy?

It is tied to the exact commit, runs in the intended trust boundary, uses reviewed dependencies and permissions, and publishes verifiable evidence rather than only a green status.

Chapter Summary

- GitHub Actions reliability begins with event and trust modeling.
- Explicit permissions and immutable dependencies reduce workflow blast radius.
- Contexts can contain attacker-controlled data and require safe handling.
- Evidence must identify exactly what source, workflow, runner, and dependencies executed.

Chapter 22: GitHub Runner Fleets, Larger Runners, Self-Hosted Runners, and Actions Runner Controller

Runners are the execution plane and primary containment boundary for GitHub Actions. A secure fleet distinguishes hosted and self-hosted use cases, separates trust tiers, uses ephemeral capacity where possible, controls network access, and measures queue, provisioning, execution, cleanup, and failure behavior.

Learning Objectives

- Compare GitHub-hosted, larger, self-hosted, and Kubernetes-based runner models.
- Design runner groups, labels, network boundaries, autoscaling, image governance, and cleanup controls.
- Explain Actions Runner Controller scale sets and their operational dependencies.
- Establish service objectives and incident controls for runner fleets.

Core Concepts

GitHub-hosted runners: Managed ephemeral virtual machines provide clean environments and low operational burden. They are appropriate for many standard workloads but require awareness of available images, included software, network egress, resource size, and platform updates.

Larger runners: Managed larger runners offer more compute, custom labels, runner groups, and selected networking capabilities. They reduce fleet operations while supporting workloads that exceed standard hosted capacity.

Self-hosted runners: Self-hosted runners provide custom hardware, network reach, software, and cost control. They also execute repository code inside organization infrastructure and therefore require strong isolation, patching, ephemeral cleanup, credential boundaries, and repository access restrictions.

Runner groups and labels: Groups control which organizations or repositories can use runners, while labels route jobs by capability. Treat both as authorization and placement controls; avoid one broad group that can reach every sensitive network.

Ephemeral model: A runner should ideally process one job and then be destroyed. Reused machines can retain credentials, source, caches, process state, containers, or filesystem changes from previous jobs.

Actions Runner Controller: ARC manages GitHub Actions runner scale sets on Kubernetes. It connects workflow demand to runner pods, but Kubernetes capacity, admission, image supply chain, pod security, network policy, persistent caches, upgrades, and GitHub authentication remain operator responsibilities.

Fleet observability: Measure queued time by label, provisioning latency, registration failures, online capacity, job duration, image age, cleanup success, pod or VM termination, saturation, and cost. A green controller view does not prove the runner plane is healthy.

Reference Runner Trust Tiers

Tier	Example Workload	Execution Model	Network and Credentials
Public validation	Fork PR lint and unit tests	Hosted or disposable isolated runner	No private network; read-only source; no environment secrets
Trusted build	Protected-branch packaging	Ephemeral hosted/larger/self-hosted	Artifact registry write only; short-lived identity
Release	Signing and production deployment	Dedicated ephemeral runner group	Restricted egress; environment approval; production role only
Specialized	GPU, mobile, hardware, private build	Dedicated self-hosted pool	Project allowlist; isolated network; measured cleanup

Architecture and Operating Decisions

- Prefer managed hosted runners unless a documented hardware, network, compliance, or performance requirement justifies self-hosting.
- Do not route untrusted fork code to persistent self-hosted runners or privileged network zones.
- Use runner groups as repository authorization boundaries and labels as capabilities, not as informal naming only.
- Build runner images through a controlled image pipeline and track image digest, tool versions, vulnerabilities, and retirement date.
- Autoscale from queue demand with maximum capacity, cost, startup-time, and failure limits; test what happens when the cloud or Kubernetes cluster cannot provision.

Production Checklist

- Each runner pool has an owner, trust tier, allowed repositories, labels, image, network policy, and capacity objective.
- Self-hosted runners are patched, ephemeral where possible, and verified clean after jobs.
- Runner authentication and registration credentials are protected and rotated.
- Queue and provisioning telemetry is available by label and repository class.
- Compromise response can isolate a runner group, revoke identities, and preserve evidence.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Jobs remain queued	No matching label, group access denied, capacity exhausted, or runner offline	Inspect requested labels, group policy, online state, and provisioning events before adding capacity	Queue reason, group policy, runner events
Self-hosted job sees previous files	Runner is persistent or cleanup failed	Quarantine the runner, rotate exposed credentials, destroy it, and move to one-job ephemeral instances	Disk image, job history, cleanup logs
ARC scale set does not create pods	Controller authentication, webhook/polling, Kubernetes quota, or image pull failure	Inspect ARC controller logs, scale-set state, cluster events, quotas, and image credentials	Controller logs, pod events, GitHub request ID
Runner cost spikes	Unbounded autoscaling, long startup, stuck jobs, or oversized class	Set maximum runners, job timeouts, right-size pools, and alert on queue-to-cost anomalies	Runner-hours, queue duration, job timeout data

Hands-on Lab 22: Design and Test a Segmented Runner Fleet

Prerequisites

- A GitHub organization or enterprise sandbox.
- One managed runner class and one disposable self-hosted or ARC-based pool.
- Two test repositories representing public validation and trusted release.
- A telemetry destination.

Implementation

1. Create separate runner groups for untrusted validation and trusted release repositories.
2. Assign labels for operating system, architecture, trust tier, and capability without encoding secrets or mutable versions.

3. Run synthetic jobs that record runner name, image identity, queue time, provisioning time, and cleanup result.
4. Attempt to route the public repository to the trusted pool and prove group policy blocks it.
5. Exhaust trusted-pool capacity with bounded jobs and observe queue behavior and autoscaling limits.
6. Destroy or recycle runners and prove no workspace or credential material remains accessible to the next job.

Validation Evidence

- Repository access restrictions are enforced.
- Queue and provisioning metrics distinguish capacity from configuration errors.
- Runner images and digests are recorded.
- Cleanup and destruction evidence is retained.

Cleanup

- Remove temporary runner registrations and cloud resources.
- Revoke temporary runner credentials and archive sanitized metrics.

Interview Questions and Model Answers

1. Why are self-hosted runners high risk?

They execute repository-controlled code inside organization infrastructure and may retain data between jobs. A compromise can reach networks, caches, credentials, and other repositories unless isolation is strong.

2. What does ARC solve?

It automates runner registration and autoscaling on Kubernetes. It does not solve cluster security, image governance, capacity, network policy, secrets, upgrades, or workload trust.

3. What is the difference between runner groups and labels?

Groups restrict which repositories or organizations may use runners. Labels describe capabilities and select eligible runners. Both must be designed deliberately.

4. Which runner metric matters most?

Queue wait and blocked reason by workload class, combined with provisioning latency and capacity. Aggregate online runner count can hide unusable or incorrectly labeled capacity.

Chapter Summary

- Runners are the execution and containment plane.
- Managed hosted capacity reduces operations but does not remove dependency and identity risk.
- Self-hosted runners require strict segmentation and ephemeral cleanup.
- ARC scales runners, while the Kubernetes platform remains an operator responsibility.

Chapter 23: Reusable Workflows, Composite Actions, Custom Actions, and Organization Delivery Products

Reusable automation can reduce duplication and create secure golden paths, but only when interfaces, versioning, dependency ownership, permission boundaries, testing, and rollout are explicit. Centralization without contracts merely spreads one hidden failure to every repository.

Learning Objectives

- Distinguish reusable workflows, composite actions, JavaScript actions, Docker actions, templates, and repository scripts.
- Design typed inputs, outputs, secrets, permissions, compatibility, and versioning for reusable delivery products.
- Test and release reusable automation with canary consumers and rollback.
- Govern dependency pinning, provenance, deprecation, and adoption across an organization.

Core Concepts

Reusable workflow: A workflow invoked with `workflow_call` can define jobs, runners, environments, permissions, inputs, secrets, and outputs. It is appropriate for end-to-end delivery stages and central policy, but caller and called workflow permission behavior must be understood.

Composite action: A composite action packages a sequence of steps for use inside a job. It cannot define independent jobs or runner selection, making it suitable for focused build, test, packaging, or evidence operations.

JavaScript or Docker action: Custom actions can provide richer logic and stable interfaces. They introduce runtime and dependency maintenance, require build and release discipline, and should not become opaque binaries that teams cannot inspect or reproduce.

Inputs and outputs: Treat reusable automation as an API. Validate input types and allowed values, expose meaningful outputs, document defaults and failure behavior, and avoid passing large unstructured JSON or secret material between layers.

Permissions and secrets: The caller should grant the minimum permissions needed. Reusable workflows should not silently assume broad tokens. Environment secrets have special behavior and should be documented and tested.

Versioning: Use immutable release references for consumers, semantic compatibility rules, changelogs, migration guides, canary repositories, and supported release windows. Moving default branches are not production release channels.

Golden path governance: A reusable product needs an owner, support model, service objective, dependency inventory, security review, adoption telemetry, deprecation plan, and escape hatch. Platform standards should reduce cognitive load without blocking justified exceptions.

Reference Reusable Workflow Contract

```
name: build-product
on:
  workflow_call:
    inputs:
      language:
        type: string
        required: true
    publish:
```

```

    type: boolean
    default: false
  secrets:
    registry-token:
      required: false
  outputs:
    artifact-digest:
      value: ${jobs.build.outputs.digest}

permissions:
  contents: read
  packages: write

jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      digest: ${steps.package.outputs.digest}
    steps:
      - uses: actions/checkout@<reviewed-sha>
      - id: package
        run: ./platform/build.sh "${inputs.language}"

```

Architecture and Operating Decisions

- Use reusable workflows for job-level orchestration and composite actions for step-level behavior.
- Keep organization delivery products small, composable, and explicit instead of one universal workflow with hundreds of switches.
- Pin every transitive action dependency and publish a dependency manifest with each release.
- Release through immutable tags or SHAs, test backward compatibility, and canary changes before broad adoption.
- Publish support, deprecation, exception, and incident procedures along with the technical interface.

Production Checklist

- Inputs, outputs, secrets, permissions, defaults, and error behavior are documented.
- Reusable automation is tested with representative caller repositories.
- Dependencies are reviewed, pinned, and monitored.
- Consumers use immutable supported releases.
- Owners can identify affected repositories and roll back a bad release.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Called workflow lacks permission	Caller token did not grant capability or reusable workflow assumed elevation	Set minimal caller permissions and document required scope; do not rely on hidden defaults	Caller and callee permission blocks, API error
Environment secret differs from caller input	Job-level environment secret overrides passed secret behavior	Document environment usage and test the exact caller/callee path	Workflow graph, environment name, secret source metadata
Central update breaks many repositories	Consumers used moving reference or compatibility was not tested	Rollback immutable release pointer and add contract/canary tests before republishing	Release tag, affected consumers, test gap

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Composite action cannot choose runner	Composite actions run inside caller job	Move job-level decisions to a reusable workflow and keep the composite focused on steps	Caller workflow and action definition

Hands-on Lab 23: Publish a Versioned Reusable Build Workflow

Prerequisites

- An organization automation repository.
- Two sample consumer repositories.
- Permission to publish tags or releases.
- A test artifact registry or local mock.

Implementation

1. Define a reusable workflow with typed inputs, a documented permission contract, and an artifact-digest output.
2. Extract one repeated step sequence into a composite action and pin its dependencies.
3. Create contract tests that call the workflow from two representative repositories and verify outputs and failure behavior.
4. Publish version 1.0.0 using an immutable reference and migrate both consumers.
5. Introduce a breaking input change on a development branch and prove compatibility tests fail.
6. Publish a backward-compatible patch, canary it on one repository, then promote it to the supported release channel.

Validation Evidence

- Consumer repositories produce the same artifact contract.
- Permissions and secrets are no broader than documented.
- The breaking change is detected before release.
- Release notes and rollback references are retained.

Cleanup

- Delete disposable artifacts.
- Retain the tagged reusable workflow and sanitized contract-test evidence.

Interview Questions and Model Answers

1. Reusable workflow or composite action?

Use a reusable workflow when you need jobs, runners, environments, permissions, or end-to-end orchestration. Use a composite action for a reusable sequence of steps inside one job.

2. How do you prevent a platform workflow from becoming a monolith?

Create small products with stable contracts, compose them, keep implementation scripts testable, and allow governed extension points instead of adding endless flags.

3. How should consumers reference reusable automation?

Use immutable reviewed releases, ideally tags backed by protected release processes or exact SHAs. Avoid default branches for production dependencies.

4. What is the platform team responsible for?

Interface stability, security, dependencies, testing, documentation, service objectives, support, adoption telemetry, deprecation, rollback, and incident response.

Chapter Summary

- Reusable CI/CD is a product and API design problem.
- Workflows and actions serve different abstraction levels.
- Versioning, dependency pinning, testing, and rollback are mandatory.
- Golden paths need ownership and support, not only shared YAML.

Chapter 24: GitHub Actions Security, OIDC, Environments, Attestations, and Untrusted Contributions

GitHub Actions security depends on the interaction of events, permissions, third-party actions, runners, secrets, environments, cloud identities, artifacts, and contributor trust. The safest design avoids long-lived credentials, isolates untrusted code, and verifies what was built before privileged release.

Learning Objectives

- Threat-model pull requests, actions, runners, tokens, secrets, artifacts, and deployments.
- Implement OIDC federation with audience and subject restrictions instead of long-lived cloud keys.
- Use environments and deployment protection rules for controlled release.
- Create and verify artifact attestations and preserve supply-chain evidence.

Core Concepts

Untrusted pull requests: Fork and contributor workflows can modify source and workflow behavior. Avoid secrets and privileged self-hosted runners, review differences between `pull_request` and `pull_request_target`, and never check out untrusted code into a privileged context without a designed boundary.

OIDC federation: GitHub can issue a short-lived identity token containing repository and workflow claims. The cloud trust policy must validate issuer, audience, repository, branch or environment, workflow reference, and other relevant claims before exchanging it for temporary credentials.

Environments: Environments can scope secrets, variables, deployment history, branch restrictions, reviewers, waiting periods, and custom protection rules. They provide a release boundary when production jobs explicitly reference them.

Artifact attestations: Attestations bind an artifact subject such as a digest to provenance claims about the workflow that produced it. Verification should occur before promotion or deployment and should enforce trusted repository, workflow, and identity expectations.

Action hardening: Review publishers, pin dependencies, minimize permissions, prevent script injection, separate build and deploy jobs, use CodeQL or other analysis for custom actions, and monitor advisories.

Secret handling: Secrets can leak through logs, command-line arguments, process tables, generated files, artifacts, caches, or compromised actions. Prefer short-lived identity and scope remaining secrets by environment and repository.

Approval evidence: Manual approval is meaningful only when the approver sees the immutable artifact digest, test and policy evidence, change summary, target environment, and rollback plan. Approval of a branch name alone is weak.

Reference OIDC Deployment Pattern

```
permissions:
  contents: read
  id-token: write
  attestations: write

jobs:
  deploy:
    environment: production
    runs-on: release-runner
    steps:
      - name: Exchange GitHub OIDC token for short-lived cloud credentials
```

```

uses: cloud-provider/oidc-login@<reviewed-sha>
with:
  role: production-deployer
- name: Verify approved artifact digest and attestation
  run: ./ci/verify-release.sh "$ARTIFACT_DIGEST"
- name: Deploy immutable digest
  run: ./ci/deploy.sh "$ARTIFACT_DIGEST"

```

Architecture and Operating Decisions

- Never expose production secrets or release runners to untrusted pull-request code.
- Use OIDC trust policies restricted to exact organizations, repositories, branches or environments, workflows, and audiences.
- Separate build, attest, approve, and deploy identities so one compromised job cannot complete the entire release.
- Require environment protection for sensitive targets and serialize deployments per environment.
- Verify artifact digest and attestation at deployment time; do not rebuild after approval.

Production Checklist

- Fork and pull-request workflows have no privileged secrets or network paths.
- Cloud trust policy validates specific OIDC claims.
- Production jobs reference protected environments.
- Artifacts are immutable and attested.
- Incident response can revoke cloud trust, disable workflows, quarantine runners, and identify affected releases.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
OIDC exchange is denied	Audience, subject, branch, environment, or workflow claim does not match trust policy	Decode the non-secret claims, compare them with policy, and narrow or correct the intended condition	Claim set, policy revision, cloud audit event
Fork PR accesses internal runner	Runner group or event trust boundary is too broad	Remove repository access, quarantine the runner, rotate credentials, and redesign the validation path	Run event, runner group policy, network logs
Deployment approval releases wrong artifact	Approval referenced mutable tag or branch rather than digest	Bind approval to immutable digest and verify it immediately before deployment	Approval record, artifact digest, registry metadata
Attestation verification fails	Artifact digest changed, provenance missing, or trusted identity mismatch	Stop deployment, inspect build and registry history, and rebuild through the approved workflow if necessary	Attestation, digest, workflow identity, registry events

Hands-on Lab 24: Deploy with GitHub OIDC, Environment Approval, and Attestation Verification

Prerequisites

- A disposable cloud account or identity provider.
- A GitHub repository with environments available.
- An artifact registry.

- A sample deployable artifact.

Implementation

1. Create a cloud role whose trust policy restricts GitHub OIDC by repository, production environment, and approved workflow reference.
2. Configure a production environment with branch restrictions and required approval.
3. Build one immutable artifact, publish its digest, and create provenance or an artifact attestation.
4. Approve and deploy the exact digest using short-lived identity; record cloud audit events.
5. Attempt the same exchange from an unapproved branch or workflow and prove denial.
6. Modify or substitute the artifact reference and prove attestation or digest verification blocks deployment.

Validation Evidence

- No long-lived cloud key exists in repository settings.
- OIDC claims and cloud audit records identify the workflow.
- Environment approval is linked to the digest.
- Tampered or unauthorized release attempts fail closed.

Cleanup

- Delete the test deployment and artifact.
- Remove temporary cloud trust and environment secrets.

Interview Questions and Model Answers

1. Why is OIDC safer than a stored cloud key?

It creates short-lived credentials only after the identity provider validates workflow claims. There is no reusable static key in repository settings or runner storage.

2. What must an OIDC trust policy restrict?

At minimum issuer, audience, organization or repository, and the intended branch, tag, environment, or workflow reference. Broad subject patterns can recreate the risk of a shared credential.

3. What does an environment protect?

It can scope deployment secrets and variables, restrict branches, require reviewers or delays, integrate custom protection rules, and preserve deployment history.

4. Does an attestation guarantee secure source code?

No. It proves claims about how an artifact was produced. Source review, dependency security, runner trust, policy, and verification are still required.

Chapter Summary

- Untrusted contribution handling is the central GitHub Actions security boundary.
- OIDC replaces static cloud secrets only when claims are tightly restricted.
- Environments control privileged release.
- Attestations and digest verification strengthen artifact identity and promotion.

Chapter 25: Advanced GitHub Actions Orchestration, Matrices, Caching, Artifacts, Deployments, and Platform Operations

Large GitHub Actions estates require more than correct YAML. They need bounded matrices, reliable data handoff, cache safety, deployment serialization, cost controls, auditability, policy, observability, and standardized operations across repositories and organizations.

Learning Objectives

- Engineer DAGs, matrices, outputs, artifacts, caches, concurrency, and manual operations safely.
- Design artifact and cache retention, integrity, and cost controls.
- Operate deployments and organization-wide workflow standards.
- Measure service objectives, usage, failures, and platform adoption.

Core Concepts

Matrix strategy: Matrices expand one job definition across operating systems, language versions, architectures, regions, or test shards. Bound the Cartesian product, use include/exclude deliberately, and decide whether fail-fast supports or harms the evidence objective.

Outputs and artifacts: Small metadata can move through job outputs; files and reports should move through artifacts or external registries. Artifact names and retention must identify source SHA and avoid collisions.

Caches: Caches optimize dependencies and tool data, not release artifacts. Keys must include lockfile or input hashes and trust boundaries. Never cache secrets, writable credentials, or untrusted build outputs into privileged jobs.

Concurrency and deployments: Validation can cancel superseded revisions; environment deployments should normally serialize. A deployment record should identify environment, artifact digest, actor, source revision, and status.

Manual operations: `workflow_dispatch` is useful for controlled recovery, rollback, or maintenance, but inputs require validation and authorization. Manual workflows must not become an unreviewed bypass around protected delivery.

Enterprise controls: Repository rulesets, allowed actions, runner groups, reusable workflows, environments, audit logs, budgets, retention, and policy provide organization-wide governance. Apply them in stages with exception and migration paths.

Operational metrics: Track run success, duration, queued time, canceled and skipped work, flaky retries, runner utilization, cache hit rate, artifact volume, cost, deployment frequency, failure rate, recovery time, and adoption of approved workflows.

Reference Matrix and Deployment Controls

```
jobs:
  test:
    strategy:
      fail-fast: false
      max-parallel: 6
    matrix:
      os: [ubuntu-latest, windows-latest]
      runtime: ['20', '22']
    runs-on: ${ matrix.os }
    steps:
      - run: ./ci/test.sh "${ matrix.runtime }"
```

```

deploy:
  needs: test
  environment: production
  concurrency:
    group: production-${{ github.repository }}
    cancel-in-progress: false
  steps:
    - run: ./ci/deploy-digest.sh "$ARTIFACT_DIGEST"

```

Architecture and Operating Decisions

- Set matrix max-parallel and explicit test coverage to avoid accidental cost and capacity explosions.
- Treat caches as disposable accelerators and artifacts as evidence; do not confuse either with a governed release registry.
- Use immutable external artifacts for deployment and retain workflow artifacts only as long as evidence policy requires.
- Create organization standards for allowed actions, runner groups, permissions, reusable workflows, and environments.
- Publish usage and reliability dashboards that teams can use to improve workflows rather than only central compliance reports.

Production Checklist

- Matrix size, max parallelism, and failure policy are bounded.
- Cache keys include lockfiles and trust boundaries.
- Artifacts and releases have distinct retention and integrity controls.
- Production concurrency and approval behavior is tested.
- Organization policies, audit events, cost, and platform SLOs are monitored.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Matrix creates hundreds of jobs	Unbounded Cartesian product or dynamic input	Limit dimensions, use include/exclude, validate generated matrices, and set max-parallel	Expanded job count, matrix input, cost estimate
Privileged job consumes poisoned cache	Cache key crosses trust boundary or restore prefix is broad	Separate cache namespaces by trust and lockfile; disable cache for privileged release if integrity cannot be proven	Cache key, source event, restored entry ID
Production jobs overlap	Concurrency group includes changing SHA or branch value	Use a stable environment and service concurrency key with cancel-in-progress false	Concurrency expression, run timeline
Costs rise without faster feedback	Duplicate workflows, oversized runners, low cache value, or excessive matrix coverage	Measure critical-path time and job value, remove duplicates, right-size, and tune concurrency	Usage export, run graph, queue and duration metrics

Hands-on Lab 25: Operate a Matrix Pipeline with Safe Cache and Production Serialization

Prerequisites

- A sample multi-runtime application.
- A GitHub repository with environments.
- An artifact registry or local mock.

- Access to workflow usage data.

Implementation

1. Create a bounded matrix across two runtimes and two operating systems with max-parallel and fail-fast policy.
2. Implement a dependency cache keyed by operating system, runtime, and lockfile hash.
3. Publish test evidence per matrix cell and aggregate a release decision without rebuilding.
4. Create a production deployment job using a stable concurrency group and protected environment.
5. Start two deployments and prove the second waits rather than overlaps or cancels the first.
6. Review run duration, queue, cache hit, artifact size, and usage; propose one evidence-based optimization.

Validation Evidence

- Matrix execution is bounded and complete.
- Cache does not cross untrusted and privileged contexts.
- One immutable artifact digest is deployed.
- Production concurrency and metrics are documented.

Cleanup

- Delete test artifacts and deployments.
- Retain the optimized workflow and sanitized metrics report.

Interview Questions and Model Answers

1. Cache or artifact?

A cache is a disposable performance optimization that may be replaced or evicted. An artifact is an intentional output or evidence object with identity and retention requirements.

2. How do you control matrix cost?

Limit dimensions, use include/exclude, cap max-parallel, run broad coverage selectively, cancel obsolete validation, and measure which matrix cells detect meaningful defects.

3. Why not use commit SHA in a production concurrency group?

Every commit would create a different group, allowing deployments to overlap. Use a stable key for the environment and service.

4. What should an Actions platform team measure?

Reliability, queue and execution time, runner capacity, cost, flaky retries, cache value, artifact volume, deployment outcomes, security exceptions, and adoption of supported golden paths.

Chapter Summary

- Advanced Actions engineering is a resource, evidence, and governance discipline.
- Matrices and caches require explicit bounds and trust separation.
- Production deployment must serialize and promote immutable artifacts.
- Enterprise operations need standards, auditability, metrics, and cost control.

Chapter 26: GitLab CI/CD Architecture, YAML, Rules, DAGs, Variables, Inputs, and Pipeline Design

GitLab CI/CD defines pipelines in .gitlab-ci.yml and included configuration. Reliable design separates pipeline creation from job selection, uses needs-based DAGs for fast feedback, validates inputs, controls variable precedence, and preserves evidence across stages and downstream pipelines.

Learning Objectives

- Explain GitLab pipeline, job, stage, needs, rule, workflow, variable, input, artifact, cache, environment, and runner responsibilities.
- Control when pipelines are created and which jobs enter them.
- Design DAGs, variables, inputs, artifacts, caches, and reports safely.
- Build a maintainable pipeline with explicit failure and evidence behavior.

Core Concepts

Pipeline and jobs: A pipeline contains jobs defined in .gitlab-ci.yml and included files. Jobs run on GitLab Runner infrastructure. Stages provide broad ordering, while needs creates an explicit DAG and can start jobs as soon as dependencies complete.

workflow: rules: Workflow rules decide whether a pipeline is created. Job rules decide whether an individual job is included and can also set attributes. Poor combinations can create duplicate branch and merge-request pipelines.

CI/CD variables: Variables come from predefined contexts, YAML, project, group, instance, schedules, triggers, manual runs, dotenv reports, and other sources with precedence. Values are strings and can be overridden; sensitive data should use a secrets provider when possible.

CI/CD inputs: Inputs provide a typed and validated interface for reusable pipeline configuration. They improve predictability compared with arbitrary free-form variables and should define allowed values and defaults.

Artifacts and reports: Artifacts pass files and evidence to later jobs or users. Structured reports integrate tests, coverage, security, dotenv, and other results. Release artifacts should still live in a governed registry.

Caches: Caches accelerate dependencies and should be keyed by lockfile and trust boundary. Runner topology and cache backend affect consistency, cost, and exposure.

Failure semantics: allow_failure, retry, interruptible, timeout, manual jobs, and when conditions alter pipeline meaning. Use them intentionally and avoid turning required controls into advisory status by accident.

Reference GitLab Pipeline Skeleton

```
workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
    - when: never

default:
  interruptible: true
  timeout: 20m

stages: [validate, build, test]

validate:
```

```

stage: validate
script: ./ci/lint.sh

build:
  stage: build
  needs: [validate]
  script: ./ci/build.sh
  artifacts:
    paths: [dist/]
    expire_in: 14 days

test:
  stage: test
  needs: [build]
  script: ./ci/test.sh
  artifacts:
    when: always
    reports:
      junit: reports/junit.xml

```

Architecture and Operating Decisions

- Use workflow rules to establish one authoritative pipeline per event and avoid duplicate branch and merge-request pipelines.
- Use needs to represent actual dependencies and shorten the critical path rather than relying on stage barriers alone.
- Use inputs for reusable configuration contracts and variables for runtime data with documented precedence.
- Keep secrets out of YAML and prefer external secret providers or short-lived identity.
- Mark only truly optional work as `allow_failure` and publish evidence on required job failures.

Production Checklist

- Pipeline creation rules are mutually understandable and tested.
- DAG dependencies match real artifact and evidence needs.
- Variable sources and precedence are documented.
- Artifacts, reports, caches, retries, and timeouts have explicit policy.
- Required checks cannot silently become optional.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Two pipelines start for one commit	Branch and merge-request rules both match	Use workflow rules to choose one pipeline source and test all event combinations	Pipeline sources, rules evaluation, commit SHA
Job is missing	rules or changes condition evaluates differently than expected	Inspect pipeline source and predefined variables; use CI lint or simulation	Rule set, variable values, lint result
Variable has unexpected value	Higher-precedence source overrides YAML value	Identify every source and replace ambiguous variables with validated inputs or protected values	Variable metadata, trigger source, pipeline config

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Downstream job cannot find files	needs artifacts or artifact path/expiry is wrong	Declare artifact dependency, validate path, and separate release registry from temporary artifacts	Job graph, artifact manifest, expiry setting

Hands-on Lab 26: Engineer a Needs-Based GitLab Merge-Request Pipeline

Prerequisites

- A disposable GitLab project.
- A sample application with lint, build, and test scripts.
- A merge request and protected default branch.
- GitLab CI lint access.

Implementation

1. Create workflow rules that produce one merge-request pipeline and one default-branch pipeline without duplication.
2. Build a needs-based DAG for lint, package, unit test, and integration test.
3. Publish JUnit evidence even when tests fail and retain package artifacts for a limited period.
4. Add a validated pipeline input for a supported runtime version in included configuration.
5. Override a similarly named variable from a higher-precedence source and document why the result changes.
6. Add the required pipeline status to merge controls and verify it is tied to the latest commit.

Validation Evidence

- Only intended pipeline sources are created.
- The DAG starts jobs as soon as dependencies complete.
- Input validation rejects unsupported values.
- Test evidence is available on failure.

Cleanup

- Remove temporary variables and artifacts.
- Retain sanitized pipeline graph and lint evidence.

Interview Questions and Model Answers

1. workflow rules or job rules?

Workflow rules decide whether a pipeline exists. Job rules decide whether a job is included and can set job attributes. Use both deliberately to avoid duplicates and surprises.

2. Why use needs?

It models real dependencies and permits jobs to run before earlier stages finish, reducing critical-path duration while keeping required artifact relationships explicit.

3. Inputs or variables?

Inputs provide a typed, validated interface for reusable configuration. Variables are flexible runtime strings with precedence and override behavior that can be harder to govern.

4. What is dangerous about allow_failure?

It can convert a required quality or security control into advisory status. Use it only when the organization accepts the risk and the UI meaning is clear.

Chapter Summary

- GitLab pipeline creation and job inclusion are separate control planes.
- needs creates explicit DAGs and faster feedback.
- Variables require precedence discipline; inputs strengthen reusable contracts.
- Artifacts, reports, and failure semantics determine whether pipeline evidence is trustworthy.

Chapter 27: GitLab Runner Executors, Fleet Security, Autoscaling, Caching, and Operations

GitLab Runner executes CI/CD jobs using shell, Docker, Kubernetes, instance, autoscaler, and other executor models. The runner fleet must be segmented by trust and capability, patched, observable, cost-controlled, and able to destroy job state after execution.

Learning Objectives

- Compare GitLab Runner executors and select them by isolation, performance, and workload requirements.
- Design protected, project, group, and instance runner boundaries with tags and access controls.
- Operate autoscaling, caches, concurrency, upgrades, registration, and cleanup.
- Respond to runner compromise and fleet capacity incidents.

Core Concepts

Runner scope: Runners can be assigned at project, group, or instance level. Broadly shared runners increase utilization but also widen trust and compromise impact. Scope and protected-branch settings are authorization controls.

Executors: Shell offers low isolation; Docker provides container isolation but depends on daemon and mount configuration; Kubernetes creates pods; instance and docker-autoscaler models provide elastic machines. Select based on threat model, startup, tooling, network, and cleanup.

Tags and protected runners: Tags route jobs to capabilities. Protected runners restrict execution to protected branches and tags. Do not rely on tags alone to prevent untrusted projects from reaching sensitive networks.

Autoscaling: Runner managers create cloud instances or Kubernetes pods based on job demand. Configure maximum capacity, idle behavior, instance lifetime, startup timeout, spot interruption, and failure backoff.

Cache backend: Shared caches may use object storage or distributed backends. Cache keys, access controls, encryption, retention, and trust namespaces determine whether performance optimization becomes a data-exposure path.

Job token and credentials: Jobs receive CI_JOB_TOKEN and other credentials according to configuration. Restrict job-token scope, avoid privileged containers, and ensure runner logs and debug options do not expose tokens.

Fleet lifecycle: Pin runner versions, test upgrades, monitor compatibility with GitLab, rotate authentication tokens, patch hosts and images, and remove stale registrations. A runner that appears online may still be unable to execute safely.

Reference Runner Fleet Classes

Pool	Executor	Trust	Use
Shared validation	Docker or Kubernetes	Untrusted to moderate	Lint, tests, build without private network
Protected build	Autoscaled VM or pod	Trusted branch only	Package and registry publication
Production deploy	Dedicated protected runner	High trust	Environment-approved deployment with short-lived identity
Specialized	Shell/instance/hardware	Dedicated project	Hardware, mobile, licensed or private-network workloads

Architecture and Operating Decisions

- Avoid shell and privileged Docker for shared untrusted workloads unless a documented containment model exists.
- Use protected runners and project or group restrictions for privileged build and deployment paths.
- Prefer one-job ephemeral machines or pods and verify destruction or cleanup after job completion.
- Separate caches by trust level and never place credentials or release artifacts in cache storage.
- Set autoscaling maximums and alert on queue, startup, failure, interruption, and cost anomalies.

Production Checklist

- Each runner fleet has scope, tags, protected status, executor, image, network, capacity, and owner documented.
- Runner authentication tokens and cloud credentials are rotated.
- Images and hosts are patched and vulnerability-scanned.
- Caches are encrypted, namespaced, retained, and access-controlled.
- Quarantine and forensic procedures exist for compromised runners.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Job is stuck	No runner matches tags, protected setting, scope, or capacity	Inspect job tags, runner scope, protected status, online state, and autoscaling events	Stuck reason, runner config, scale events
Docker job can control host	Privileged mode or Docker socket exposed	Quarantine runner, rotate credentials, remove dangerous mount, and use isolated build architecture	Runner config, mount list, job image, audit logs
Autoscaler creates no instances	Cloud permission, quota, image, network, or manager config failure	Inspect runner manager logs and cloud events; validate quota and template independently	Manager logs, API error, instance template
Cache leaks across projects	Shared key or bucket policy is too broad	Namespace by project and trust, restrict credentials, invalidate exposed cache, and review jobs	Cache key, bucket access logs, affected pipelines

Hands-on Lab 27: Build and Load-Test a Secure GitLab Runner Fleet

Prerequisites

- A GitLab group sandbox.
- A Docker, Kubernetes, or autoscaled runner manager.
- Cloud or cluster capacity for disposable runners.
- Metrics and log collection.

Implementation

1. Create separate shared-validation and protected-release runner configurations with distinct scopes and tags.
2. Run an unprotected branch job and prove it cannot use the protected release runner.
3. Configure bounded autoscaling and generate concurrent jobs while measuring queue and provisioning time.
4. Enable a shared dependency cache with project and lockfile namespacing, then prove another project cannot restore it.

5. Simulate runner termination during a job and confirm retry or failure behavior preserves evidence.
6. Upgrade the runner manager in a staged manner and verify registration, execution, cache, and cleanup after the change.

Validation Evidence

- Protected runner restrictions are enforced.
- Autoscaling remains inside maximum capacity.
- Cache access is isolated.
- Metrics distinguish queue, provisioning, execution, and interruption.

Cleanup

- Delete disposable runners and cached objects.
- Revoke temporary runner and cloud credentials.

Interview Questions and Model Answers

1. Which GitLab Runner executor is safest?

There is no universal answer. Ephemeral VMs or well-hardened Kubernetes pods often provide strong isolation, while shell is weak for shared workloads. Security depends on host, network, mounts, credentials, lifecycle, and trust.

2. Why are protected runners useful?

They restrict sensitive runners to protected branches or tags, reducing the chance that unreviewed code reaches privileged networks or credentials.

3. What should autoscaling monitor?

Queue wait, provisioning latency and failures, active and idle capacity, interruption, job duration, cleanup, cloud quota, and cost by workload class.

4. How do you respond to a runner compromise?

Stop scheduling, quarantine or destroy the runner, revoke tokens and cloud identities, preserve logs and images when required, identify jobs and artifacts, invalidate caches, and review downstream releases.

Chapter Summary

- GitLab Runner is a high-value execution plane.
- Executor selection must follow the threat model.
- Scope, protected status, tags, network, and identity form the placement boundary.
- Autoscaling and caching require both cost and security controls.

Chapter 28: GitLab Includes, CI/CD Components, Parent-Child Pipelines, and Multi-Project Delivery

GitLab supports reusable configuration through includes and CI/CD components, and decomposes delivery through parent-child and multi-project pipelines. These features create scalable platform products only when contracts, versioning, permissions, status propagation, evidence, and ownership are explicit.

Learning Objectives

- Design local, project, remote, template, and component includes safely.
- Publish typed CI/CD components with versioned interfaces.
- Use child and multi-project pipelines to decompose monorepos and orchestrate systems.
- Control dependency trust, status propagation, artifacts, and release compatibility.

Core Concepts

Includes: GitLab can include local files, other projects, templates, remote files, and components. Included configuration is executable dependency content and must be pinned, reviewed, and governed.

CI/CD components: A component is a reusable pipeline configuration unit with defined inputs. Components can package jobs or larger pipeline segments and are suitable for organization catalogs when released and tested like software.

Parent-child pipelines: A parent pipeline can generate or trigger child pipelines, enabling monorepo decomposition, dynamic configuration, and independent subsystem graphs. Artifact and status relationships must be designed explicitly.

Multi-project pipelines: One project can trigger a pipeline in another project for platform, infrastructure, or coordinated release. Permissions, branch selection, token scope, status strategy, and version compatibility determine safety.

Dynamic child pipeline: A job can generate YAML and trigger it as a child pipeline. Generated configuration must be treated as code, validated, bounded, and protected from untrusted data injection.

Contract and versioning: Inputs, outputs, artifact names, variable requirements, permissions, failure behavior, and supported versions form the component API. Use immutable releases and compatibility tests.

Status and evidence: Decide whether downstream failure fails the parent, whether strategy mirror or dependent behavior is required, and how evidence is aggregated. A green parent must not hide a failed required child.

Reference Component and Child Pipeline

```
# Component consumer
include:
  - component: $CI_SERVER_FQDN/platform/ci/build@1.2.0
  inputs:
    runtime: "22"
    publish: true

# Parent-child trigger
service_a:
  trigger:
    include: services/a/.gitlab-ci.yml
    strategy: mirror
  rules:
```

```
- changes: [services/a/**/*]
```

```
# Pin component releases; avoid moving default branches in production.
```

Architecture and Operating Decisions

- Use components for supported reusable products and ordinary includes for repository structure that does not require a public contract.
- Pin cross-project and remote configuration to immutable versions or commit SHAs.
- Use child pipelines to separate monorepo domains and multi-project pipelines for independently owned repositories.
- Validate generated YAML and cap the number of dynamic jobs or children.
- Make required downstream status visible in the parent and aggregate evidence by source revision and release identity.

Production Checklist

- Reusable configuration has an owner, version, changelog, compatibility tests, and support window.
- Cross-project dependencies are immutable and authorized.
- Child and downstream pipeline status propagation matches policy.
- Generated configuration is validated and bounded.
- Affected consumers can be identified and rolled back.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Included config changes unexpectedly	Moving branch or remote URL was used	Pin the dependency, restore prior release, and add policy against mutable production includes	Resolved config, dependency reference, release history
Parent is green while child failed	Trigger strategy or allow-failure semantics do not propagate status	Use the required trigger strategy and test failure propagation	Parent/child graph, trigger config, status API
Child pipeline is empty	Rules evaluate in child context differently than expected	Inspect pipeline source and inherited variables; test the child configuration directly	Child source, rule values, compiled YAML
Multi-project trigger is denied	Job token allowlist, role, branch, or trigger permission mismatch	Use the narrowest supported token and configure explicit project access	Token scope, project settings, API response

Hands-on Lab 28: Publish a GitLab CI/CD Component and Monorepo Child Pipelines

Prerequisites

- A GitLab group with a component project and consumer project.
- A sample monorepo with two services.
- Permission to create releases or tags.
- CI lint access.

Implementation

1. Create a build component with typed inputs, test fixtures, and a versioned release.
2. Consume the component from both services using an immutable release reference.
3. Create child pipelines triggered only when each service path changes and propagate required status to the parent.
4. Generate one bounded dynamic child pipeline from a validated service manifest.

5. Publish a breaking component change on a development reference and prove compatibility tests fail.
6. Canary a backward-compatible release, then update consumers and document rollback.

Validation Evidence

- Component inputs and outputs are documented.
- Only changed services create child pipelines.
- Parent status reflects required child failures.
- Consumers can roll back to the prior component release.

Cleanup

- Delete temporary releases and dynamic artifacts not required for evidence.
- Retain sanitized component catalog and compatibility results.

Interview Questions and Model Answers

1. Include or component?

Use a component when you need a supported reusable interface with typed inputs and release lifecycle. Use includes for internal configuration composition that does not need a product contract.

2. Child or multi-project pipeline?

Use child pipelines for decomposition within one repository and source revision. Use multi-project pipelines when independently owned repositories or release processes must coordinate.

3. Why is generated YAML risky?

Untrusted data can alter the pipeline graph or commands. Validate the generator inputs, restrict allowed templates, cap size, and lint the result before triggering.

4. How should downstream status work?

Required downstream failures must fail or block the parent according to policy. The final status should not hide incomplete or failed release work.

Chapter Summary

- GitLab reusable configuration is an executable dependency.
- Components need typed contracts and versioned releases.
- Child and multi-project pipelines solve different ownership boundaries.
- Status, artifacts, and evidence must propagate deliberately.

Chapter 29: GitLab Pipeline Security, ID Tokens, Job Tokens, Protected Environments, and Deployment Safety

GitLab CI/CD security joins pipeline sources, variables, secrets, runners, job tokens, OIDC ID tokens, environments, approvals, artifacts, and downstream projects. Secure delivery limits identity duration and scope, separates untrusted validation from privileged release, and protects the exact environment and artifact being changed.

Learning Objectives

- Threat-model variables, runners, includes, images, job tokens, ID tokens, artifacts, and deployments.
- Use id_tokens for OIDC federation and narrow cloud trust policies.
- Control CI_JOB_TOKEN access and cross-project interactions.
- Use protected environments, deployment approvals, resource groups, and immutable artifacts for safe release.

Core Concepts

ID tokens: Jobs can request OIDC ID tokens with the id_tokens keyword and a chosen audience. External trust policies should validate issuer, audience, project, namespace, ref, environment, pipeline source, and other claims before granting temporary credentials.

CI_JOB_TOKEN: The job token authenticates to selected GitLab features with constrained scope related to the triggering user and project settings. Configure allowlists and avoid using personal access tokens when a narrower job token or OIDC path exists.

Variables and secrets: Masked or hidden variables reduce accidental display but do not make an unsafe job trustworthy. Variables can be overridden or exfiltrated by malicious scripts. Prefer external secrets and protected variables for trusted refs.

Protected environments: Protected environments restrict who or what can deploy. Combine them with protected branches or tags, approvals where available, and dedicated protected runners.

Resource groups: Resource groups serialize deployments or other stateful jobs targeting the same resource. They are essential when overlapping pipelines could corrupt an environment.

Images and includes: Container images, components, templates, and included YAML execute as supply-chain dependencies. Pin immutable digests or releases, scan them, and restrict who can update central configuration.

Deployment evidence: Record project and commit, pipeline ID, job, runner, environment, artifact digest, approval, temporary identity, policy results, deployment status, and rollback decision.

Reference Secure GitLab Deployment

```
deploy_production:
  stage: deploy
  environment:
    name: production
  resource_group: production-api
  id_tokens:
    CLOUD_ID_TOKEN:
      aud: https://cloud.example.com
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
  script:
    - ./ci/exchange-oidc.sh "$CLOUD_ID_TOKEN"
```

```
- ./ci/verify-artifact.sh "$ARTIFACT_DIGEST"
- ./ci/deploy.sh "$ARTIFACT_DIGEST"
```

Pair with protected environment, protected runner, and narrow cloud trust.

Architecture and Operating Decisions

- Use short-lived OIDC identity for cloud access and validate precise claims.
- Restrict job-token cross-project access with allowlists and use the narrowest supported token.
- Keep protected variables and production runners unavailable to unprotected refs and merge-request code.
- Serialize environment changes with resource groups and deploy immutable digests.
- Require approval and evidence for the artifact being deployed, not just the pipeline or branch.

Production Checklist

- OIDC audiences and trust claims are narrow and tested negatively.
- Job-token project access is allowlisted.
- Protected environments, refs, variables, and runners align.
- Images and included configuration are pinned.
- Deployment and rollback evidence is retained outside ephemeral runner storage.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Cloud identity exchange fails	ID token audience or claims do not match trust policy	Inspect claims and cloud policy, then correct the intended narrow condition	Token claims without signature secret, policy version, audit event
Job token cannot access downstream project	Allowlist or role does not permit it	Configure explicit project access or use an approved trigger/token mechanism	Job token scope setting, API response
Unprotected branch reads production variable	Variable protection or ref protection is misconfigured	Revoke exposed secret, protect the variable, and test negative access from unprotected refs	Variable setting, pipeline source, audit event
Two deployments collide	No resource group or unstable key	Use one stable resource group per environment and service; design cancel/retry behavior	Job timeline, resource group, environment events

Hands-on Lab 29: Release to a Protected GitLab Environment with OIDC and Serialized Deployment

Prerequisites

- A GitLab project with protected branches and environments.
- A disposable cloud identity provider.
- A protected runner.
- An immutable sample artifact.

Implementation

1. Configure a protected production environment and restrict deployment to an approved role.
2. Create a protected runner and prove unprotected branch jobs cannot use it.
3. Request an ID token with an explicit audience and configure a cloud trust policy for the project, default branch, and environment.
4. Deploy an immutable artifact digest through a stable resource group and retain cloud audit evidence.

5. Start a second production job and prove it waits instead of overlapping.
6. Attempt identity exchange from an unprotected branch and prove denial; then revoke the temporary trust.

Validation Evidence

- Production variable and runner access are restricted.
- OIDC exchange succeeds only for the approved context.
- Deployment jobs serialize.
- Audit records identify project, pipeline, job, and environment.

Cleanup

- Delete the test environment and artifact.
- Revoke cloud trust and temporary variables.

Interview Questions and Model Answers

1. Masked variable or external secret?

Masked variables reduce accidental display but remain available to job code. External secret retrieval with short-lived identity provides better rotation, audit, and scope for sensitive values.

2. What is CI_JOB_TOKEN for?

It gives a running job constrained access to supported GitLab resources. Its scope is narrower than a personal token but still requires allowlists, runner security, and leak prevention.

3. Why use resource groups?

They serialize state-changing jobs for the same resource, preventing overlapping deployments or migrations from corrupting an environment.

4. What should a protected deployment require?

Trusted ref, protected runner, protected environment, least-privilege short-lived identity, immutable artifact, policy and test evidence, approval where required, serialization, verification, and rollback.

Chapter Summary

- GitLab security depends on aligned refs, runners, variables, identities, and environments.
- ID tokens enable short-lived federation with precise claims.
- Job-token scope must be controlled.
- Resource groups and immutable artifacts make deployments safer.

Chapter 30: Cross-Platform CI/CD Platform Operations and Enterprise Capstone

Enterprises often operate GitHub Actions, GitLab CI/CD, and legacy systems simultaneously. The goal is not identical YAML. The goal is a consistent delivery contract: trusted source, reproducible build, tested artifact, least-privilege identity, protected environment, observable execution, recoverable deployment, and evidence that can survive platform migration.

Learning Objectives

- Define a platform-neutral delivery contract and map it to GitHub Actions and GitLab CI/CD.
- Establish governance, service objectives, telemetry, cost controls, support, and migration practices.
- Design platform outage, runner compromise, identity failure, and artifact incident response.
- Complete an enterprise cross-platform CI/CD capstone.

Core Concepts

Delivery contract: Define source identity, change review, workflow version, runner trust, dependency lock, test evidence, artifact digest, SBOM, attestation, approval, deployment identity, environment, verification, and rollback independent of vendor syntax.

Golden paths: Provide GitHub reusable workflows and GitLab components that implement the same control intent while respecting platform-native features. Avoid lowest-common-denominator abstractions that hide useful controls.

Policy and exceptions: Control allowed dependencies, permissions, runners, tokens, environments, artifact registries, retention, and identity. Exceptions need owner, rationale, compensating controls, expiry, and review.

Observability: Normalize run ID, platform, repository or project, commit SHA, workflow or pipeline revision, runner class, stage, artifact digest, environment, deployment, result, duration, queue, retry, and cost into a common telemetry model.

Service objectives: Set objectives for platform availability, critical validation latency, runner provisioning, deployment success, artifact publication, secrets access, and recovery. Measure separately for public validation, trusted build, and production release.

Migration: Migrate by delivery contract and workload class. Run both systems for limited qualification, compare artifacts and evidence, freeze mutable behavior, cut over triggers, and define rollback to the prior platform.

Incident response: Maintain playbooks for compromised dependency, runner breach, stolen token, OIDC trust error, artifact tampering, platform outage, duplicate deployment, missing evidence, and central workflow regression.

Platform product: Publish supported use cases, onboarding, templates, reusable products, runner tiers, service levels, ownership, support, cost model, roadmap, deprecation, and adoption metrics.

Platform-Neutral Release Manifest

```
release:
  source_revision: <commit-sha>
  pipeline_platform: github-actions | gitlab-ci
  pipeline_definition_revision: <sha-or-release>
  runner_class: trusted-build
  build_environment_digest: sha256:<digest>
  artifact_digest: sha256:<digest>
```

```

sbom_digest: sha256:<digest>
attestation_reference: <uri>
tests:
  required: passed
policies:
  required: passed
target_environment: production
deployment_identity: <federated-role>
approval_reference: <id>
rollback_reference: <previous-digest>

```

Architecture and Operating Decisions

- Standardize outcomes and evidence, not every implementation detail.
- Keep release artifacts and evidence in systems independent of ephemeral CI/CD workspaces.
- Provide equivalent trust tiers and identity models across platforms.
- Run migration qualification using the same source revision and compare artifact digest, tests, SBOM, and deployment result.
- Design central reusable products so a bad release can be canaried, paused, and rolled back without editing every repository manually.

Production Checklist

- A platform-neutral release contract is documented and machine-readable.
- GitHub and GitLab golden paths implement equivalent required controls.
- Runner tiers, identities, environments, dependencies, and artifact registries have owners.
- Unified telemetry supports reliability, security, cost, and adoption analysis.
- Outage and compromise playbooks are tested and migration rollback is possible.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Same source produces different artifacts	Environment, dependencies, timestamps, architecture, or build script differs	Compare build manifests, lockfiles, images, clocks, and scripts; remove nondeterminism before migration	Both manifests, artifact diff, environment digests
Central workflow release breaks many teams	Insufficient compatibility testing or moving references	Freeze release, rollback consumer pointer, identify affected repositories, and expand canary coverage	Release ID, consumers, failure pattern, rollback evidence
Platform outage blocks production fix	No alternate release path or offline evidence	Use the approved emergency path with preserved controls and reconcile source after recovery	Incident approval, artifact digest, deployment audit, reconciliation PR
Unified dashboard shows misleading success	Platforms map canceled, manual, skipped, or downstream states differently	Define a normalized status model and preserve native state for drill-down	Mapping specification, sample runs, query results

Hands-on Lab 30: Build, Break, Recover, and Migrate an Enterprise CI/CD Product

Prerequisites

- One GitHub repository and one GitLab project containing the same sample service.
- A shared artifact registry.
- Disposable cloud environment with OIDC trust for both platforms.

- A central telemetry or evidence store.

Implementation

1. Define a machine-readable release contract and implement one GitHub reusable workflow and one GitLab component that produce it.
2. Build the same source revision on both platforms using pinned environments and compare artifact digest, SBOM, tests, and attestation.
3. Deploy one approved digest through protected environments using platform-specific OIDC identities and normalized evidence.
4. Introduce a compromised reusable dependency or runner misconfiguration in a sandbox and exercise detection, containment, credential revocation, and affected-release analysis.
5. Simulate a platform outage and execute the approved recovery or alternate-path procedure without rebuilding the artifact.
6. Cut over the authoritative trigger from one platform to the other, validate production behavior, and demonstrate rollback to the prior platform.

Validation Evidence

- Both platforms emit the same release contract.
- One immutable digest is promoted and verified.
- The security incident preserves evidence and identifies impact.
- Migration and rollback complete within measured objectives.

Cleanup

- Remove temporary identities, runners, environments, and artifacts.
- Retain sanitized architecture, manifests, incident report, and migration report.

Interview Questions and Model Answers

1. How do you standardize GitHub Actions and GitLab CI/CD?

Define a common delivery contract and required controls, then implement them using each platform native features. Standardize evidence and outcomes rather than forcing identical syntax.

2. How do you migrate CI/CD platforms safely?

Pin dependencies, qualify both platforms on the same source, compare artifacts and evidence, cut over triggers in a controlled window, preserve the prior path, and define rollback.

3. What should remain outside the CI/CD platform?

Governed release artifacts, long-term evidence, policy definitions, identity systems, production observability, and recovery documentation should survive a platform outage.

4. What makes CI/CD a platform product?

Supported golden paths, clear ownership, service objectives, runner and identity tiers, security controls, onboarding, telemetry, cost model, support, roadmap, and deprecation lifecycle.

Chapter Summary

- Cross-platform operations should standardize delivery contracts and evidence.
- Golden paths must preserve native platform strengths.
- Unified telemetry needs a carefully defined state model.
- Migration and incident response must avoid rebuilding approved artifacts.

Appendix A: Ten Additional Enterprise Portfolio Projects

- 1. Organization Actions Governance Baseline:** Implement allowed-action policy, explicit token permissions, runner groups, rulesets, environments, reusable workflows, audit export, and exception lifecycle.
- 2. GitHub ARC Runner Platform:** Build an isolated Kubernetes runner scale-set platform with pod security, network policy, image pipeline, queue SLO, autoscaling limits, cleanup, and compromise response.
- 3. Reusable Workflow Product Catalog:** Publish versioned build, test, package, sign, attest, and deploy workflows with typed contracts, canary consumers, dependency manifests, and deprecation policy.
- 4. GitHub OIDC Multi-Cloud Deployment:** Federate GitHub Actions to AWS, Azure, or Google Cloud using environment- and workflow-restricted claims and common release evidence.
- 5. GitLab Component Catalog:** Create tested CI/CD components for multiple languages with inputs, release channels, compatibility matrices, consumer discovery, and rollback.
- 6. GitLab Autoscaled Runner Fleet:** Operate protected and untrusted runner pools with autoscaler, cache backend, cost controls, observability, staged upgrades, and clean-room recreation.
- 7. Monorepo Child Pipeline Factory:** Generate bounded child pipelines by changed service, aggregate test/security evidence, and propagate required status to one release decision.
- 8. Cross-Project Release Orchestrator:** Coordinate application, infrastructure, database, and security projects with narrow tokens, immutable releases, environment approvals, and failure recovery.
- 9. CI/CD Evidence Lake:** Normalize GitHub and GitLab run, runner, artifact, attestation, deployment, approval, and cost events into searchable audit and reliability datasets.
- 10. Hosted CI/CD Migration Factory:** Create assessment, dual-run qualification, artifact comparison, cutover, rollback, and decommission playbooks for migrating Jenkins or another platform to GitHub or GitLab.

Appendix B: Senior Interview Drill Bank

1. Design a GitHub Actions platform for 2,000 repositories.

Cover repository classes, reusable workflows, runner groups, hosted versus self-hosted capacity, OIDC, environments, allowed actions, telemetry, cost, support, deprecation, and incident response.

2. A fork pull request compromised a self-hosted runner. What now?

Stop scheduling, quarantine or destroy runners, revoke tokens and cloud identities, preserve evidence, identify jobs/artifacts/caches, inspect downstream releases, fix group policy, and test the new boundary.

3. GitHub Actions builds are slow only during office hours.

Analyze queue by label, hosted concurrency limits, self-hosted provisioning, matrix expansion, cache performance, duplicate workflows, critical path, and cost-effective capacity.

4. A reusable workflow update broke 400 repositories.

Freeze and rollback the release, identify consumers, compare compatibility tests, communicate impact, restore service, then improve canary, immutable references, versioning, and change policy.

5. Design GitLab CI/CD for a large monorepo.

Use workflow rules, changes-based child pipelines, components, needs DAGs, artifact contracts, runner tags, cache namespace, evidence aggregation, and one controlled release decision.

6. GitLab job tokens can access too many projects.

Inventory access, restrict allowlists, prefer project-scoped or OIDC identities, rotate exposed credentials, test negative access, and monitor cross-project API calls.

7. How do you choose GitHub Actions versus GitLab CI/CD?

Evaluate source platform, governance, runner requirements, reusable ecosystem, identity, security, deployment, audit, self-managed needs, cost, skills, and migration risk rather than feature count alone.

8. How do you prove one artifact moved through environments?

Build once, identify by digest, attach SBOM and provenance, record approvals and policy, promote the same digest, verify at deployment, and retain environment deployment records.

9. How would you respond to a compromised third-party action?

Block the reference, identify all workflow runs and artifacts that used it, revoke credentials, quarantine runners, assess releases, replace or pin a safe version, and add dependency policy and monitoring.

10. What is a good CI/CD platform SLO?

Use user-visible outcomes such as critical validation latency, runner provisioning, artifact publication, deployment success, and recovery, segmented by trust and workload class.

11. How do you avoid cache poisoning?

Namespace by trust, repository, OS, runtime, and lockfile; restrict writer contexts; avoid privileged restore prefixes; never cache secrets or release outputs; invalidate on incident.

12. How do you make manual dispatch safe?

Restrict who can invoke it, validate inputs, use protected environments and identities, preserve immutable artifact references, and ensure it cannot bypass required policy or evidence.

13. How do you normalize GitHub and GitLab statuses?

Define common states such as queued, running, passed, failed, canceled, skipped, manual, blocked, and degraded while retaining native details and downstream relationships.

14. How do you recover when the hosted CI/CD platform is unavailable?

Use pre-approved emergency procedures, external artifact and evidence stores, independent deployment tooling or alternate platform, strict audit, and reconciliation after recovery.

15. What is the difference between CI/CD governance and bureaucracy?

Governance defines risk-based controls, evidence, ownership, and exceptions. Bureaucracy adds steps without measurable risk reduction or feedback. Use metrics and remove ineffective gates.

Appendix C: Command and Investigation Reference

```
# GitHub CLI examples
gh workflow list
gh workflow view <workflow>
gh run list --workflow <workflow>
gh run view <run-id> --log
gh run rerun <run-id> --failed
gh api repos/<owner>/<repo>/actions/runs/<run-id>
gh api repos/<owner>/<repo>/actions/artifacts

# Local GitHub workflow static checks (examples)
yamllint .github/workflows
actionlint

# GitLab CLI/API examples
glab ci lint
```

```

glab pipeline list
glab pipeline view <pipeline-id>
glab job trace <job-id>
curl --header "PRIVATE-TOKEN: $TOKEN" "$CI_API_V4_URL/projects/<id>/pipelines/<pipeline-id>"

# GitLab Runner operations
gitlab-runner verify
gitlab-runner list
gitlab-runner status
gitlab-runner unregister --name <runner>

# Evidence and artifact checks
sha256sum artifact.tar.gz
cosign verify-attestation <image-or-artifact-reference>
syft <artifact-or-image> -o spdx-json > sbom.json

# OIDC investigation
# Decode JWT header/claims without treating it as authorization evidence.
python - <<'PY2'
import base64, json, os
for part in os.environ['OIDC_TOKEN'].split('.')[2]:
    part += '=' * (-len(part) % 4)
    print(json.dumps(json.loads(base64.urlsafe_b64decode(part)), indent=2))
PY2

```

Appendix D: Production-Readiness Checklist

- Repository and project classes have documented trust levels and supported workflow patterns.
- Workflow and pipeline events are bounded and duplicate runs are controlled.
- Token permissions and job identities are least privilege and short lived.
- Third-party actions, components, images, and includes are reviewed, pinned, and monitored.
- Reusable workflows and components have typed contracts, versioning, testing, support, and rollback.
- Hosted and self-hosted runner pools have owners, trust tiers, authorization, labels/tags, images, network, capacity, and cleanup policy.
- Untrusted contributions cannot access private networks, production credentials, protected runners, or deployment environments.
- OIDC trust validates exact issuer, audience, repository/project, ref, environment, and workflow/pipeline claims.
- Secrets are minimized, scoped, rotated, and excluded from logs, artifacts, caches, and support bundles.
- Matrices, fan-out, concurrency, retries, and timeouts are bounded.
- Caches are disposable, namespaced by trust, and never contain release artifacts or credentials.
- Artifacts are immutable, identified by digest, scanned, signed or attested where required, and promoted without rebuilding.
- Environment deployment is protected, serialized, approved where required, verified, and recoverable.
- Required checks are tied to the exact commit and downstream status is not hidden.
- Runner queue, provisioning, execution, cleanup, failure, interruption, saturation, and cost are monitored.
- Workflow/pipeline success, duration, retries, cancellations, flaky tests, artifact volume, deployment outcomes, and DORA metrics are available.
- Audit logs and evidence are retained outside ephemeral runners according to policy.
- Platform outages, compromised dependencies, runner breaches, token leaks, OIDC trust errors, cache poisoning, and bad central releases have tested runbooks.

- Exceptions have owner, rationale, compensating controls, expiry, and review.
- The organization can migrate or fail over delivery without rebuilding an already approved artifact.

Appendix E: Glossary

Action: Reusable executable logic used as a GitHub Actions step.

Actions Runner Controller: A Kubernetes operator and controller pattern for managing GitHub Actions runner scale sets.

Artifact: A file or evidence output retained from a workflow or pipeline; governed release artifacts should use an external registry.

Attestation: A signed statement that binds an artifact subject to provenance or other claims.

Cache: Disposable stored data intended to accelerate jobs, not serve as release evidence.

CI/CD component: A reusable GitLab pipeline configuration unit with inputs and a release lifecycle.

CI_JOB_TOKEN: A token issued to a GitLab job for constrained access to supported GitLab resources.

Composite action: A GitHub action composed of multiple steps that runs inside the caller job.

Concurrency group: A GitHub Actions key that limits simultaneous workflow runs or jobs.

Environment: A named deployment target with variables, secrets, history, and protection controls.

Executor: The GitLab Runner mechanism used to execute jobs, such as shell, Docker, Kubernetes, or autoscaled instances.

GitHub-hosted runner: An ephemeral managed runner provided by GitHub.

ID token: An OIDC token requested by a CI/CD job for exchange with an external identity provider.

Include: A GitLab mechanism for composing pipeline configuration from another file, project, template, remote source, or component.

Job: A unit of CI/CD work executed on one runner.

Matrix: A GitHub Actions strategy that expands one job definition across parameter combinations.

needs: A dependency declaration that forms a job DAG in GitHub Actions or GitLab CI/CD.

OIDC: OpenID Connect, used to exchange short-lived CI/CD identity claims for external credentials.

Parent-child pipeline: A GitLab pipeline relationship used to decompose one project or monorepo.

Protected environment: A GitLab environment whose deployment access is restricted.

Protected runner: A GitLab runner limited to protected branches or tags.

Reusable workflow: A GitHub Actions workflow invoked by another workflow through workflow_call.

Resource group: A GitLab control that serializes jobs using the same named resource.

Runner group: A GitHub authorization boundary controlling which repositories may use a runner set.

Runner scale set: A group of GitHub Actions runners managed and scaled as one pool, commonly through ARC.

workflow: rules: GitLab configuration that determines whether a pipeline is created.

Appendix F: Official Documentation Index

The following primary sources form the technical baseline for this volume. Verify current platform plans, feature availability, runner images, release notes, security advisories, and API behavior before production changes.

- [GitHub Actions documentation](#)
- [GitHub workflow syntax](#)
- [GitHub contexts reference](#)
- [GitHub concurrency](#)
- [GitHub reusable workflows](#)
- [GitHub runner reference](#)
- [GitHub self-hosted runners](#)
- [Actions Runner Controller](#)
- [GitHub secure use reference](#)
- [GitHub OIDC](#)
- [GitHub deployments and environments](#)
- [GitHub artifact attestations](#)
- [GitHub billing and usage](#)
- [GitLab CI/CD pipelines](#)
- [GitLab CI/CD YAML reference](#)
- [GitLab CI/CD inputs](#)
- [GitLab CI/CD components](#)
- [GitLab downstream pipelines](#)
- [GitLab Runner](#)
- [GitLab Runner advanced configuration](#)
- [GitLab Runner autoscaling](#)
- [GitLab Runner security](#)
- [GitLab pipeline security](#)
- [GitLab OIDC ID tokens](#)
- [GitLab CI/CD job token](#)
- [GitLab environments](#)
- [GitLab protected environments](#)
- [GitLab deployment safety](#)

Volume 10C Completion Standard

You have completed Part C when you can design, secure, scale, observe, recover, and migrate GitHub Actions and GitLab CI/CD platforms; publish supported reusable delivery products; operate segmented runner fleets; deploy with short-lived identity and protected environments; and prove that one reviewed source revision produced one immutable artifact that can be promoted and recovered across platform boundaries.

- **Build:** Produce the same release contract from GitHub Actions and GitLab CI/CD.
- **Defend:** Prove untrusted changes cannot reach protected runners, secrets, private networks, or production roles.
- **Verify:** Pin dependencies, capture SBOM and provenance, and verify artifact digest before deployment.
- **Operate:** Measure queue, runner, workflow, pipeline, cache, artifact, deployment, cost, and recovery behavior.
- **Recover:** Contain a runner or dependency compromise and restore safe delivery without rebuilding an approved artifact.
- **Migrate:** Qualify both platforms, cut over one delivery path, and demonstrate rollback with complete evidence.